University of Exeter

ECM2433: The C Family

Lecture 02: Crash Course in Rust

Variables: `let` and `mut`

```
1 let x = 5;        // immutable by default
2 x = 6;            // error!
3
4 let mut y = 5;    // mutable
5 y = 6;            // ok
```

- Variables are immutable by default
- Add `mut` to allow mutation
- Opposite of C, where everything is mutable unless `const`

- [Book ch 3.1]

University of Exeter

Shadowing and `const`

```rust
let x = 5;
let x = x + 1;        // shadowing: x is now 6
let x = "hello";      // can change type: x is now a string

let mut y = 10;
let y = y;            // can change mutability: y is now immutable

const MAX_POINTS: u32 = 100_000;  // compile-time constant
```

- Shadowing: redeclare with `let` to rebind a name
- Can change type or mutability when shadowing
- `const`: compile-time constant, requires type annotation
- [Book ch 3.1]

University of Exeter

Integer Types

```
1 let a: i8 = -5;      // signed 8-bit: -(2^7) to (2^7)-1 (-128 to 127)
2 let b: u8 = 255;     // unsigned 8-bit: 0 to (2^8)-1 (0 to 255)
3 let c = 42;          // i32 by default
4 let d = 1_000_000;   // underscores for readability
```

- Overflow: panics in debug builds, wraps in release
- [Book ch 3.2]

Signed	Unsigned	Size
i8	u8	8-bit
i16	u16	16-bit
i32	u32	32-bit
i64	u64	64-bit
isize	usize	pointer-sized

University of Exeter

Other Scalars: `float`, `bool`, `char`

```
1 let x = 2.0;          // f64 by default
2 let y: f32 = 3.0;     // f32 if specified
3
4 let t = true;
5 let f: bool = false;
6
7 let c = 'z';
8 let crab = '🦀';      // char is 4 bytes (Unicode scalar value)
```

- `f64` is the default float (also `f32`)
- `bool`: `true` or `false`
- `char`: single quotes, 4 bytes, full Unicode support
- [Book ch 3.2]

University of Exeter

Compound Types: Tuples and Arrays

```
1 let tup: (i32, f64, u8) = (500, 6.4, 1);
2 let (x, y, z) = tup;          // destructuring
3 let five_hundred = tup.0;     // index access
4
5 let arr: [i32; 5] = [1, 2, 3, 4, 5];  // type: [type; length]
6 let first = arr[0];
7 let arr2 = [3; 5];            // [3, 3, 3, 3, 3]
```

- Tuples: fixed size, mixed types, access with `.0`, `.1`, etc.
- Arrays: fixed size, same type, access with `[index]`
- Array bounds are checked at runtime (panics on out-of-bounds)

- [Book ch 3.2]

University of Exeter

Printing Values: `println!`, `{}`, `{:?}`

```rust
let x = 42;
let name = "Rust";
println!("x is {}", x);                // x is 42
println!("{} has {} types", name, x); // Rust has 42 types

let tup = (1, 2.0, 'a');
println!("{:?}", tup);                 // (1, 2.0, 'a')
println!("debug: {x:?}");              // debug: 42
```

- `{}`: display format (most types)
- `{:?}`:debug format (tuples, arrays, and more)
- Can inline variable names: `{x}`, `{x:?}`
- More options: https://doc.rust-lang.org/std/fmt/

University of Exeter

Functions and Parameters

```
1 fn add(x: i32, y: i32) -> i32 {
2     x + y            // implicit return (no semicolon)
3 }
4
5 fn subtract(x: i32, y: i32) -> i32 {
6     return x - y;    // explicit return
7 }
```

- `fn` keyword to define functions
- Parameters must have type annotations
- Return type after `->`
- Can return implicitly (expression) or explicitly (`return`)
- [Book ch 3.3]

University of Exeter

Expressions vs Statements

```
1 let y = {
2     let x = 3;
3     x + 1        // expression: no semicolon, returns value
4 };               // y is 4
5
6 let z = {
7     let x = 3;
8     x + 1;       // statement: semicolon, returns ()
9 };               // z is ()
```

- Statements perform actions, don't return values
- Expressions evaluate to a value
- Semicolon turns an expression into a statement
- Blocks `{ }` are expressions; they `return` their last value
- [Book ch 3.3]

University of Exeter

Return Values and Unit Type

```rust
fn five() -> i32 {
    5
}

fn print_value(x: i32) {
    println!("{}", x);
}   // returns () implicitly

fn explicit_unit() -> () {
    println!("hello");
}
```

- Functions without `->` `return ()` implicitly
- `->  ()` can be written explicitly but is usually omitted
- Return type must match the final expression
- [Book ch 3.3]

University of Exeter

Comments

```rust
// This is a line comment

let x = 5; // can go at end of line

/// This is a doc comment for the following item
/// It supports Markdown formatting
fn documented_function() {
    // ...
}
```

- `//` for line comments
- `///` for doc comments (generates documentation)
- Doc comments go before the item they document
- [Book ch 3.4, 14.2]

University of Exeter

```
hello_rust
fn documented_function()
```

This is a doc comment for the following item It supports Markdown formatting

`if` Expressions

```rust
let number = 5;

if number > 0 {
    println!("positive");
} else if number < 0 {
    println!("negative");
} else {
    println!("zero");
}
```

```rust
let number = 5;

let description = if number > 0 {
    "positive"
} else if number < 0 {
    "negative"
} else {
    "zero"
};
```

- No parentheses around condition (unlike C)
- Condition must be `bool` (no truthy/falsy)
- `if` is an expression: can return a value
- All branches must return the same type

University of Exeter

[Book ch 3.5]

Loops: `loop`, `while`, `for`

```rust
loop {
    println!("forever");
    break;                  // exit the loop
}

let mut n = 3;
while n > 0 {
    println!("{}", n);
    n -= 1;
}

for i in 1..5 {             // 1, 2, 3, 4 (exclusive end)
    println!("{}", i);
}

for i in 1..=5 {            // 1, 2, 3, 4, 5 (inclusive end)
    println!("{}", i);
}
```

- `loop`: infinite `loop`, exit with `break`
- `while`: conditional `loop`
- `for`: iterate over a range or collection
- `1..5` excludes 5; `1..=5` includes 5
- [Book ch 3.5]

University of Exeter

Loop Control: labels, `break`, `continue`

```rust
'outer: for i in 0..3 {
    for j in 0..3 {
        if i == j {
            continue;           // skip to next j
        }
        if i + j > 3 {
            break 'outer;       // exit outer loop
        }
        println!("{}, {}", i, j);
    }
}

let mut counter = 0;
let result = loop {
    counter += 1;
    if counter == 10 {
        break counter * 2;      // return value from loop
    }
};  // result is 20
```

- `continue`: skip to next iteration
- `break`: exit the `loop`
- Loop labels: target a specific `loop`
- `break` can return a value from `loop`
- [Book ch 3.5]

University of Exeter

Recap

- Variables: `let`, `mut`, shadowing, `const`
- Types: integers (overflow checked in debug!), `float`, `bool`, `char`, tuples, arrays
- Functions: parameters, return types, expressions vs statements, unit type `()`
- Comments: `//` and `///` for documentation
- Control flow: `if` expressions, `loop`/`while`/`for`, labels, `break`/`continue`

- Required reading: Book Chapter 3

University of Exeter